

SQL Análisis de Seguridad: Filtrado de Amenazas y Control de Datos

Descripción del proyecto

En este proyecto, utilicé consultas SQL para investigar posibles incidentes de seguridad mediante el análisis de registros de acceso y datos de empleados. Mi objetivo fue filtrar y recuperar información específica de las tablas `log_in_attempts` y `employees` para identificar vulnerabilidades y segmentar datos para actualizaciones de sistema. A través de esta actividad, demuestro mi capacidad para aplicar lógica de filtrado compleja y asegurar la integridad de la información organizacional.

Recuperar después de horas de intentos fallidos de inicio de sesión

```
MariaDB [organization]> SELECT *
->
-> FROM log_in_attempts
->
-> WHERE login_time > '18:00' AND success = 0;
+-----+-----+-----+-----+-----+-----+
| event_id | username | login_date | login_time | country | ip_address |
| success |
+-----+-----+-----+-----+-----+-----+
|          |          |            |            |         |            |
+-----+-----+-----+-----+-----+-----+
```

Para investigar un posible incidente ocurrido fuera del horario laboral, filtré los registros de acceso para identificar intentos fallidos realizados después de las 18:00.

- **Consulta:** `SELECT * FROM log_in_attempts WHERE login_time > '18:00' AND success = 0;`
- **Explicación:** Esta consulta filtra datos de **fecha y hora** utilizando el operador mayor que (>) para identificar registros posteriores a las '18:00'. Además, utiliza el operador lógico **AND** para asegurar que se cumplan dos condiciones simultáneamente: que el acceso sea tardío y que el campo `success` sea `0` (indicando un fallo).

Recuperar intentos de inicio de sesión en fechas específicas

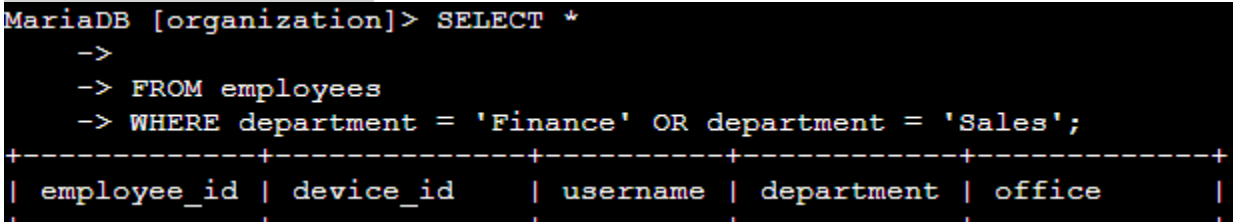
Investigué un evento sospechoso ocurrido entre el 8 y el 9 de mayo de 2022 mediante el análisis de fechas de acceso.

- **Consulta:** `SELECT * FROM log_in_attempts WHERE login_date = '2022-05-08' OR login_date = '2022-05-09';`

```
MariaDB [organization]> SELECT *
->
-> FROM log_in_attempts
->
-> WHERE login_date = '2022-05-08' OR login_date = '2022-05-09';
```


Recuperar empleados en Finanzas o Ventas

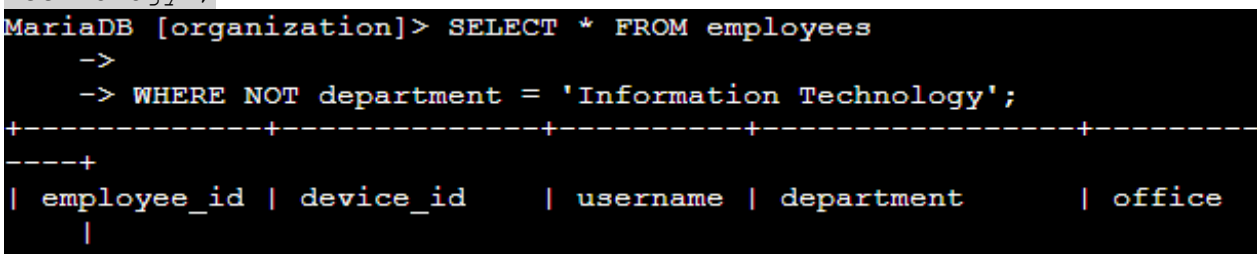
Segmenté la información para actualizaciones en los departamentos de Ventas y Finanzas.

- **Consulta:** `SELECT * FROM employees WHERE department = 'Finance' OR department = 'Sales';`


```
MariaDB [organization]> SELECT *
->
-> FROM employees
-> WHERE department = 'Finance' OR department = 'Sales';
+-----+-----+-----+-----+-----+
| employee_id | device_id | username | department | office |
+-----+-----+-----+-----+-----+
| 1 | 1 | Alice | Finance | 101 |
```
- **Explicación:** Utilicé el operador **OR** para capturar empleados de ambos departamentos. Fue necesario especificar la columna `department` para ambas condiciones para que la lógica de unión funcionara correctamente.

Recuperar a todos los empleados que no pertenecen al departamento de TI.

Finalmente, identifiqué a los empleados que no pertenecen al equipo técnico para completar las actualizaciones pendientes.

- **Consulta:** `SELECT * FROM employees WHERE NOT department = 'Information Technology';`


```
MariaDB [organization]> SELECT * FROM employees
->
-> WHERE NOT department = 'Information Technology';
+-----+-----+-----+-----+-----+
| employee_id | device_id | username | department | office |
+-----+-----+-----+-----+-----+
| 1 | 1 | Alice | Finance | 101 |
```
- **Explicación:** El operador **NOT** se utilizó para excluir a los empleados del departamento de TI, permitiendo enfocar los recursos en el resto de la organización.

Resumen

En este proyecto, apliqué una serie de filtros SQL avanzados para investigar incidentes de seguridad y gestionar actualizaciones de sistemas. Mediante el uso estratégico de operadores como **AND**, **OR**, **NOT** y patrones con **LIKE**, logré extraer información precisa de grandes conjuntos de datos. Estas habilidades son fundamentales para un analista de seguridad que busca proteger los activos de información mediante el análisis detallado de registros y activos.